

A Fast Splendor Environment for Self-Play Reinforcement Learning

Aditya Kumar

Carnegie Mellon University adityaku@andrew.cmu.edu

September 2026

Abstract

This note describes an implementation of the board game Splendor as a reinforcement learning environment. The rules are implemented exactly, for two to four players, in a small C library that runs about four million turns per second on one core. Around it we built a self-play trainer with a pool of past opponents, an Elo ladder, a Monte Carlo tree search, and a browser interface. A policy trained for 300 million steps on one desktop beats a strong greedy heuristic by about 230 Elo. Adding 64 simulations of search per move adds about 100 more. Code and trained models are at <https://github.com/Sylindril/splendor>.

1 Why Splendor

Splendor sits between the classic two-player board games and large multi-agent simulations. It has hidden cards, random draws, decisions inside a turn (which tokens to return, which noble to take), and two to four players. Games last about sixty turns. We could not find an implementation fast enough to train on hundreds of millions of samples, so we wrote one.

2 The environment

The game logic is one C header. A struct holds one game. Each seat is an agent, so a vector of N games exposes NP agents to the learner, where P is the number of players. One step is one turn by the seat to move. The other seats are idle for that step: their actions are ignored and their observation says so.

The rules are the published ones. Ninety cards in three tiers, four face up per tier. Ten nobles, of which $P + 1$ are dealt. A bank of 4, 5 or 7 gems per color for 2, 3 or 4 players, plus five gold. On a turn a player takes up to three gems of different colors, takes two of one color if at least four remain, reserves a card and gains a gold, or buys a card. Cost is paid from bonus cards first, then gems, then gold. A player above ten tokens returns tokens one at a time. After a purchase, a qualifying noble visits; if several qualify, the player chooses. The game ends at the end of the round in which someone reaches fifteen points. Fewest cards breaks ties. The starting seat is random. The card list was checked against four other open implementations.

The observation is a vector of $240 + 48P$ bytes: the bank, deck sizes, the twelve face-up cards, the nobles, one block per player (tokens, bonuses, points, reserved cards; opponents' deck-top reserves are hidden), a 72-way legal-action mask, the turn number, and a flag saying whether this seat is to move. The policy reads the mask from the observation and masks its own logits. The trainer never sees an illegal action. The engine can also snapshot and restore a game, freeze a game for a step, and re-deal everything a given seat cannot see. The search uses these.

On an Apple M4 Max, one core runs 4.1 million turns per second with random actions, or 1.9 million when actions are sampled from the mask in NumPy. One million random steps at each player count run clean under the address and undefined-behavior sanitizers. Tests check that tokens, cards and nobles are conserved, that nobody holds more than ten tokens outside a discard phase, and that illegal actions change nothing.

3 Training

The policy is a three-layer MLP of width 512 with LayerNorm, 0.74M parameters, with an actor head and a value head. Training is PPO as implemented in PufferLib [1].

Plain self-play, where every seat is the current policy, works but has two costs. Half or more of the samples come from idle seats, and the policy can forget how to beat older versions of itself. A wrapper fixes both. It shows the trainer only one seat per game. When the learner acts, the wrapper plays every other seat with a frozen policy on the CPU, batched across games, until it is the learner’s turn again. Every sample is now a decision.

The frozen opponents come from two places. Half the games use the latest weights, reloaded from disk every few seconds. The rest use a checkpoint from a pool, drawn with the prioritized fictitious self-play weights of Vinyals et al. [2]: member i is picked in proportion to $(1 - w_i)^2 + 0.05$, where w_i is a running estimate of the learner’s win rate against it. A checkpoint joins the pool every twenty million steps. Rewards are +1 for a win, −1 for a loss, and 0.02 per prestige point.

The wrapper runs about 180 thousand learner steps per second per process on the reference machine. With eight workers and the GPU, training runs at 150 to 180 thousand samples per second. The 300 million step run in the next section took 34 minutes.

4 Evaluation

We rate agents with the Bradley–Terry model, fit by the MM algorithm of Hunter [3], with draws counted as half a win each way and a one-game prior so that undefeated agents get a finite rating. Ratings are reported on the Elo scale, with a uniform-random agent fixed at zero. Two-player ladders are round robins. Ladders for three and four players draw random tables and turn each final ranking into pairwise results.

A greedy heuristic serves as a fixed reference. It buys the most valuable card it can afford, otherwise takes the gems that bring the nearest card closest. It beats random every time.

Two players, 100 games per pair	Elo	Win %
League, 300M steps	1210	75.7
League pool, 240M	1197	74.5
League pool, 160M	1049	58.0
Plain self-play, same network, 60M	994	52.4
Greedy heuristic	982	51.0
League pool, 80M	945	46.8
League pool, 40M	740	28.9
Random	0	6.7

Table 1: Two-player ladder. The curve flattens after 240M steps but has not stopped.

Four players, 150 random tables	Elo
League, 200M steps	1017
League pool, 175M	1009
League pool, 125M	971
Greedy heuristic	894
League pool, 75M	886
League pool, 25M	699
Random	0

Table 2: Four-player ladder. The run was shorter and each seat sees a quarter of the decisions, so the margin over the heuristic is smaller.

The league policy is about 230 Elo above the greedy bot and 215 above plain self-play with the same network. We have no human ratings, so we make no claim about human strength. The interface lets a person play the models directly.

5 Search

The search is PUCT over engine snapshots, batched across games. In each round every root walks to a leaf; one vectorized environment step expands all the leaves; one forward pass evaluates them. Each leaf is evaluated from every seat’s point of view, so the backed-up value is a vector and works for more than two players. Terminal leaves use the engine’s own rewards. Before searching, the root is re-dealt from the point of view of the seat to move, so the search does not see hidden cards. The engine’s random state is part of the snapshot, so one search explores one deal.

With the 512-unit network the search runs about 26 thousand simulations per second on CPU at a batch of 64 games. Sixty-four simulations per move beat the raw policy in 63% of games, which the ladder reports as about 100 Elo. An AlphaZero-style loop [4] is included: self-play with search, then training on visit counts and outcomes. We ran it only at small scale.

6 Interface

The browser interface draws a square table with a mat for each player. Every move is animated from an event the server emits: a bought or reserved card flies to the buyer’s mat, the replacement flips in from the deck, gems move between the bank and the mats. A finished game stays on the table with the standings. The user can seat any mix of people, bots, checkpoints, and search agents, and can play with the keyboard.

7 Limitations

The ratings are relative to bots and to past checkpoints, not to people. The reward includes a small shaping term for points. The search uses one deal per move. All runs used one desktop.

References

- [1] J. Suarez. PufferLib: Making reinforcement learning libraries and environments play nice. *arXiv:2406.09411*, 2024.

- [2] O. Vinyals et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575:350–354, 2019.
- [3] D. R. Hunter. MM algorithms for generalized Bradley–Terry models. *Annals of Statistics*, 32(1):384–406, 2004.
- [4] D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.